



CENTRO UNIVERSITÁRIO INTERNACIONAL UNINTER

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

**RAIZES DO NORDESTE: BACKEND MULTICANAL
DE UMA REDE DE LANCHONETES NORDESTINAS**

LUCAS DOS ANJOS DE SOUSA – RU: 4998832

CARAPICUÍBA – SÃO PAULO

2026

Prof. Me. Luciane Yanase Kanashiro

SUMARIO

1 INTRODUÇÃO E OBJETIVOS	4
2 ANÁLISE E REQUISITOS	5
2.1 Requisitos Funcionais (RF)	5
2.2 Requisitos Não Funcionais (RNF)	6
2.3 Multicanalidade como Requisito de Domínio	7
3 DIAGRAMAS	9
3.1 Diagrama de Casos de Uso	9
3.2 Diagrama Entidade-Relacionamento (DER)	11
3.3 Diagrama de Classes (Domínio)	13
3.4 Diagrama de Sequencia do Fluxo Critico	14
3.5 Diagrama de Atividade do Fluxo Critico	15
3.6 Arquitetura em Camadas	16
4 API E ENDPOINTS	18
4.1 Modulo de autenticação.....	17
4.2 Modulo de Usuários	18
4.3 Modulo de Unidades (Stores)	19
4.4 Modulo de Produtos, Estoques e Pedidos	20
4.5 Modulo de Fidelidade (Loyalty)	22
4.6 Modulo de Promoções	22
4.7 padrões de Erro e Modulo de Auditoria	22
5 LGPD, PRIVACIDADE E SEGURANCA NO BACK-END	23
6 ENTREGA TECNICA	25
6.1 Repositório Git	25
6.2 README de Execução	25
6.3 Documentação da API (Swagger/OpenAPI)	26
6.4 Coleção de Testes Postman/Insomnia	26
6.5 Banco de Dados e Modelo (DER)	26
6.6 Linha de comando e qualidade de código	26
7 PLANO DE TESTES	27

7.1 Cenários Mínimos (T01 a T25)	27
7.2 Cobertura dos Requisitos Obrigatórios	29
7.3 Testes automatizados	29
8 CONCLUSÃO	31
REFERÊNCIAS	32

1 INTRODUÇÃO E OBJETIVOS

A presente Atividade Prática tem como objetivo avaliar a capacidade do estudante de analisar um estudo de caso, modelar dados, definir contratos de API e implementar uma solução de Back-end que atenda aos principais requisitos funcionais e não funcionais do cenário proposto. O foco da trilha de Back-end é demonstrar domínio sobre o levantamento e priorização de requisitos, a modelagem do domínio e da base de dados, a organização do projeto em camadas, a implementação de uma API REST com contratos bem definidos, a persistência em banco de dados, as regras de negócio do fluxo crítico, a integração simulada de pagamento (mock), a segurança e a LGPD, bem como a testabilidade da aplicação.

O cenário proposto consiste em uma rede de lanchonetes nordestinas em expansão, ficticiamente denominada Raízes do Nordeste, cuja operação ocorre em múltiplas unidades (matriz e franquias) e atende diferentes canais de venda, a saber: App, Web, Totem, Balcão (In-Store) e Pickup. A solução de back-end foi projetada para servir de plataforma comum a todos esses canais, tratando o canal de origem como um atributo de domínio obrigatório do pedido, viabilizando a consolidação e o acompanhamento operacional por canal.

A implementação priorizou o fluxo crítico recomendado no roteiro (Fluxo A): a cadeia Pedido -> Pagamento mock -> Atualização de status, acrescida de módulos transversais de autenticação (com perfis de permissão), gestão de Unidades, Cardápio por Unidade, Controle de Estoque (global e por loja), programa de fidelização (com consentimento LGPD), Promoções (cupons) e Auditoria de ações sensíveis. Foi também implementado um mecanismo de Idempotência no fluxo de pagamento, requisito não explicitamente cobrado, mas essencial para garantir a robustez da operação em caso de retentivas de clientes ou falhas de rede.

O sistema foi implementado em NestJS (Node.js 24+), com TypeScript, Prisma ORM como camada de persistência e PostgreSQL como banco de dados relacional. A documentação da API é gerada automaticamente via Swagger/OpenAPI e disponibilizada no endpoint /docs. Os testes compreendem 125 testes unitários e 22 testes end-to-end (e2e), totalizando 147 cenários automatizados, além de uma coleção Postman executável com 25 cenários suplementares. Toda a entrega está documentada em repositório Git público, conforme exige o roteiro.

2 ANÁLISE E REQUISITOS

2.1 Requisitos Funcionais (RF)

Os requisitos funcionais abaixo foram elicitados a partir do estudo de caso e priorizados segundo o critério de valor para o MVP da atividade prática. Todos foram implementados e estão documentados neste trabalho.

- RF01 - Cadastro de usuários: o sistema deve permitir o cadastro público de clientes com coleta de CPF, consentimento LGPD e criação automática do registro de Customer associado a User.
- RF02 - autenticação(login): o sistema deve autenticar usuários por e-mail e senha, retornando access token (JWT) e refresh token (JWT separado), com persistência do hash do refresh token para revogação.
- RF03 - Renovação de sessão (refresh): o sistema deve permitir a troca de access token a partir de um refresh token válido e não revogado.
- RF04 - Logout (revogação): o sistema deve permitir o logout com revogação do refresh token, invalidando o token atual no lado servidor.
- RF05 - Reset de senha: o sistema deve prover mecanismo de solicitação e confirmação de redefinição de senha, com tokens de uso único e expiração.
- RF06 - Perfis e autorização: o sistema deve suportar os perfis ADMIN, STAFF e CUSTOMER, com garcom (WAITER), cozinha (KITCHEN) e caixa (CASHIER) como subperfis de STAFF, aplicados via Guards e Permission Guard.
- RF07 - Cadastro de unidades: o sistema deve permitir a operação CRUD de unidades (lojas) e a visualização do cardápio por unidade com a disponibilidade de estoque.
- RF08 - Cadastro de produtos: o sistema deve permitir a operação CRUD de produtos, com preço e descrição.
- RF09 - Controle de estoque: o sistema deve gerenciar estoque global e por loja, com operações de entrada, saída, transferência entre lojas e validação/pré-criação de pedido.
- RF10 - Realização de pedidos: o sistema deve criar pedidos a partir de uma unidade, um cliente autenticado, uma lista de itens com quantidade, congelando o preço no momento da criação e registrando o canalPedido (channel) de origem.

- RF11 - Atualização do status do pedido: o sistema deve permitir a transição entre PENDING, IN_KITCHEN, READY, DELIVERED e CANCELLED, com registro de auditoria.
- RF12 - Cancelamento de pedido: o sistema deve permitir o cancelamento de pedido, com devolução de estoque a unidade quando aplicável.
- RF13 - Pagamento via serviço externo (mock): o sistema deve simular um gateway de pagamento, persistindo requestPayload e responsePayload, retornando um identificador de pagamento e status coerente (SUCCESS ou FAILED), integrando-se ao fluxo do pedido.
- RF14 - Idempotencia no pagamento: o sistema deve garantir que a mesma chave de idempotencia (Header Idempotency-Key) aplicada sobre o mesmo pedido retorne o estado atual sem novo lançamento, e que o reuso da chave em outro pedido seja bloqueado com erro 409.
- RF15 - Fidelização (pontos): o sistema deve creditar pontos ao cliente (10% do total do pedido) após a confirmação de pagamento e permitir o resgate simples com geração de cupom, mediante consentimento explícito.
- RF16 - Histórico de fidelidade: o sistema deve expor um extrato de movimentacoes de pontos (EARN, REDEEM, ORDER_PAID).
- RF17 - Promoções e cupons: o sistema deve aceitar couponCode no pedido, validar o vínculo cliente-promoção (CustomerPromotion), marcar o cupom como usado após o pedido e aplicar o desconto no total.
- RF18 - Consulta e filtragem de pedidos: o sistema deve listar pedidos com filtros (canalPedido, status, cliente, unidade) e paginação.
- RF19 - Auditoria de ações sensíveis: o sistema deve registrar em tabela própria (AuditLog) as ações de criação/alteração de pedido, pagamento aprovado/recusado e crédito/resgate de pontos.

2.2 Requisitos Não Funcionais (RNF)

- RNF01 - autenticação autorização por perfil: todas as rotas, exceto login, cadastro público e endpoints públicos de leitura, exigem JWT valido e permissão explícita por papel.
- RNF02 - Hashing de senhas: as senhas sao armazenadas exclusivamente como hash bcrypt, com salt configurado pela variável SALT_ROUNDS (padrão 10).

- RNF03 - Não exposição de dados sensíveis: as respostas são entregues via View Models, garantindo que campos como password, refreshToken em texto plano e CPF não sejam serializados.
- RNF04 - Logs e auditoria: ações sensíveis (criação de pedido, mudança de status, pagamento aprovado/recusado, resgate de pontos) são registradas em AuditLog com o userId do autor e detalhes em JSON.
- RNF05 - padrão de erro consistente: respostas de erro seguem um payload padronizado com código, mensagem, detalhes, timestamp e path (vide seção 4.7).
- RNF06 - Documentação automática via Swagger/OpenAPI em /docs, com DTOs tipados e respostas exemplificadas.
- RNF07 - Persistência real: todos os fluxos usam Prisma ORM conectado a PostgreSQL real, sem simulações em memória para fluxos produtivos.
- RNF08 - Tolerância a falhas na integração de pagamento: uso do mock service com idempotencia e regras claras para retentativas.
- RNF09 - Isolamento de banco para testes e2e: o ambiente de teste usa um banco proprio (DATABASE_URL de .env.test) e um globalSetup que executa migrations e truncate de tabelas, sem afetar o banco de desenvolvimento.
- RNF10 - Codificação e formatação: o código é escrito em TypeScript com strict mode, formatado por Prettier e verificado por ESLint.

2.3 Multicanalidade como Requisito de Domínio

Conforme exigido pelo roteiro, o canal de origem do pedido (canalPedido) é tratado como dado obrigatório de domínio e persistido no pedido, garantindo a rastreabilidade operacional entre canais. No projeto, esse atributo foi modelado como o enum Channel, com os valores WEB, APP, TOTEM, IN_STORE e PICKUP, equivalentes aos canais APP, WEB, TOTEM, BALCAO e PICKUP do roteiro. O sistema valida o preenchimento do campo na criação do pedido (retornando 422 quando ausente) e expõe um endpoint de consulta com filtro por canal via query param.

O Quadro 1 a seguir mapeia os requisitos de multicanalidade do roteiro para a implementação feita.

Quadro 1 - Mapeamento dos requisitos de multicanalidade

Requisito do Roteiro	Implementacao no Projeto
Pedido com campo canal Pedido ENUM (APP, TOTEM, BALCAO, PICKUP, WEB).	Enum Channel no Prisma com valores WEB, APP, TOTEM, IN STORE, PICKUP.
Criação de pedido exige preenchimento do campo.	DTO CreateOrderDto exige o campo channel; sem ele, retorna 422.
API permite consultar/filtrar pedidos por canal.	GET /orders?channel=VALOR retorna apenas pedidos do canal informado.
Rastreabilidade operacional entre canais.	Tabela Order armazena o channel, e o canal e exibido em todas as representacoes do pedido.

3 DIAGRAMAS

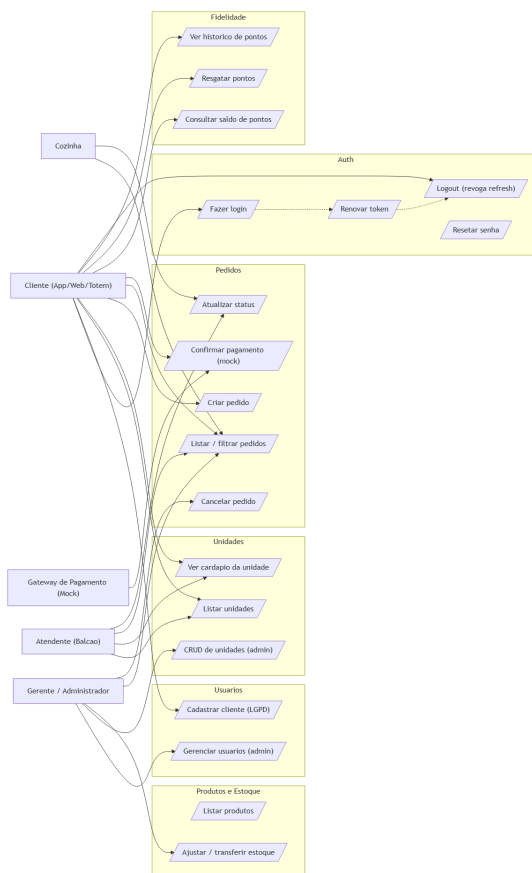
Os diagramas apresentados a seguir foram gerados a partir do schema Prisma, dos casos de uso implementados e do fluxo crítico do projeto. A representação segue a sintaxe Mermaid, que pode ser renderizada diretamente em ferramentas como GitHub, VS Code (com extensão Markdown Preview Mermaid Support), GitLab e mermaid.live. A imagem estatica do DER esta disponivel em docs/DER.png, no repositório.

3.1 Diagrama de Casos de Uso

O diagrama de casos de uso, apresentado na Figura 1, elabora os atores identificados no roteiro (Cliente App/Web/Totem, Atendente, Cozinha, Gerente/Administrador e Sistemas Externos de Pagamento) relacionados aos casos de uso do sistema, agrupados por modulo.

Figura 1 - Diagrama de Casos de Uso

[Clique para visualizar em alta resolução](#)



A Figura 2 detalha os casos de uso criticos do fluxo do pedido (UC-CRIT-1 - Criar Pedido e UC-CRIT-2 - Confirmar Pagamento Mock + Idempotencia), com pre-condicoes, pos-condicoes, fluxo principal, excecoes e regras de negócio.

Figura 2 - Descricao dos Casos de Uso Criticos

UC-CRIT-1 - Criar Pedido

Campo	Descricao
Ator principal	Cliente (App/Web/Totem)
Pré-condições	Cliente autenticado (JWT); unidade e produtos existem; há estoque disponivel na unidade.
Pós-condições	Pedido criado em status PENDING; estoque da unidade debitado; AuditLog registrado.
Fluxo principal	1. Cliente envia POST /orders com storeId, channel, items[]. 2. Sistema valida unidade e produtos. 3. Sistema valida estoque por item. 4. Sistema calcula total e cria o pedido. 5. Sistema debita StoreStock. 6. Sistema responde 201 com pedido criado.
Exceções	404 unidade/produto inexistente; 409 estoque insuficiente; 422 payload invalido; 401/403 sem permissao.
Regras de negócio	channel obrigatorio; quantidade > 0; preco congelado no item no momento da criacao.

UC-CRIT-2 - Confirmar Pagamento (Mock + Idempotencia)

Campo	Descricao
Ator principal	Cliente (App/Web/Totem)
Atores secundários	Gateway de Pagamento (Mock interno)
Pré-condições	Pedido existe e está em PENDING ou IN_KITCHEN; cliente envia header Idempotency-Key.
Pós-condições	Pedido movido para DELIVERED (apos confirmacao); pontos creditados; Payment e AuditLog registrados.
Fluxo principal	1. Cliente envia PATCH /orders/:id com { confirmPayment: true } e header Idempotency-Key. 2. Sistema consulta IdempotencyKey. 3. Se existir para o mesmo pedido, retorna estado atual. 4. Senao, chama MockPaymentService.requestPayment (deterministico pelo prefixo da chave). 5. Em sucesso: atualiza status, credita 10% do total em pontos, grava idempotency key. 6. Em falha: grava idempotency key com 409, retorna 409 PAYMENT FAILED.
Exceções	409 IDEMPOTENCY_KEY_CONFLICT se a chave pertence a outro pedido; 409 PAYMENT_FAILED se mock rejeitar.

Regras de negócio	Mesma chave + mesmo pedido = replay (sem novo pagamento); prefixo approve- aprova; prefixo reject-rejeita; sem prefixo = fallback 50%.
--------------------------	--

3.2 Diagrama Entidade-Relacionamento (DER)

O DER do projeto foi modelado em coerencia com o schema Prisma (prisma/schema.prisma) e esta representado na Figura 3, em notacao Mermaid ER. Sao 15 entidades, com chave primaria em todas (string UUID) e chaves estrangeiras explicitas. As cardinalidades principais sao: User 1:1 Customer (um usuario pode ser cliente; Customer.userId UNIQUE); Customer 1:N Order (um cliente tem varios pedidos); Store 1:N Order e Store 1:N StoreStock (cada combinacao storeId+productId UNIQUE em StoreStock); Product 1:1 GlobalStock e 1:N StoreStock; Order 1:N OrderItem, 1:N Payment e 1:N IdempotencyKey; Promotion 1:N CustomerPromotion (cada par cliente+promocao UNIQUE). As restricoes relevantes sao: preco congelado no OrderItem, estoque proprio por unidade, pagamento desacoplado (Payment separado do Order, com request/response em JSON), e idempotencia via IdempotencyKey com TTL de 24 horas.

Figura 3 - Diagrama Entidade-Relacionamento

[Clique para visualizar em alta resolução](#)

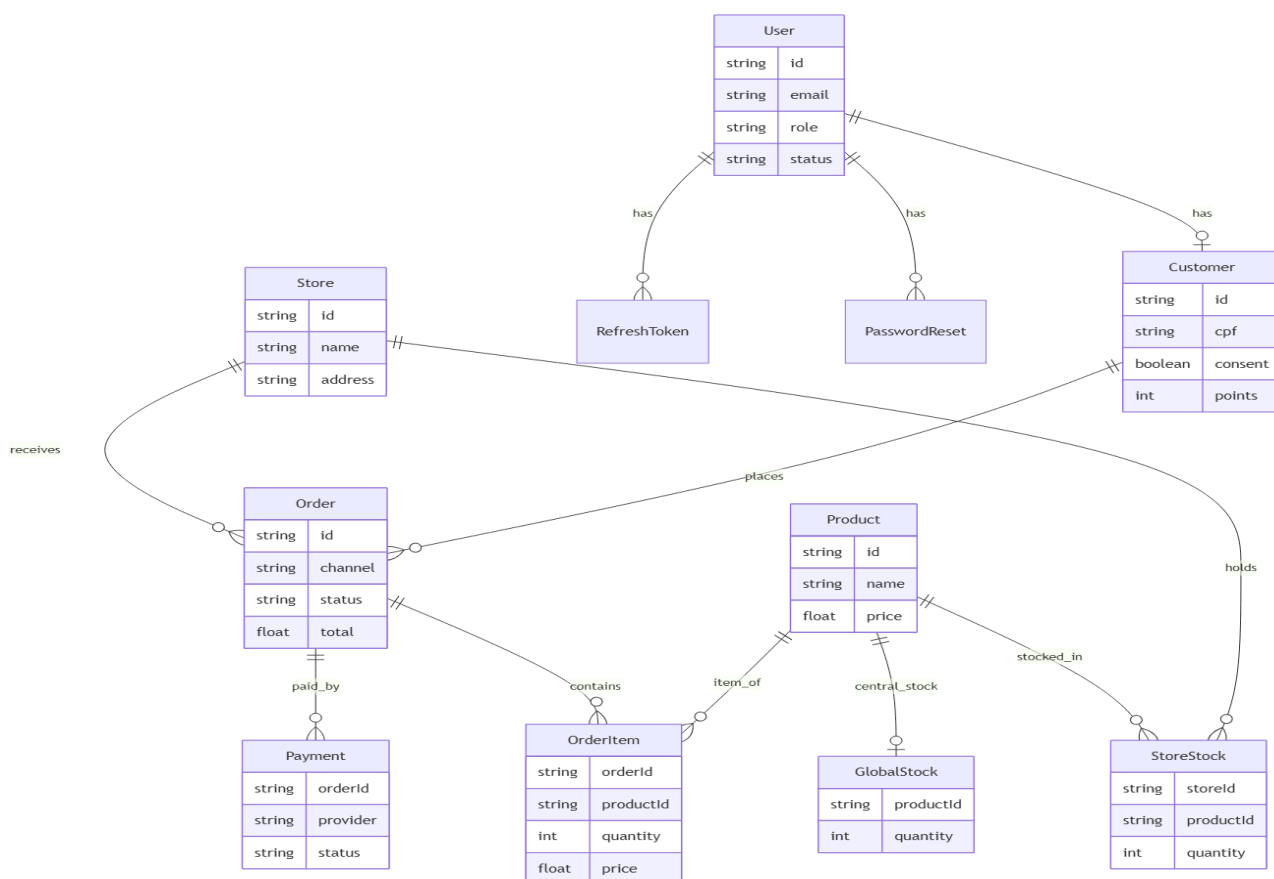


Figura 4 - Cardinalidades e Restricoes do DER

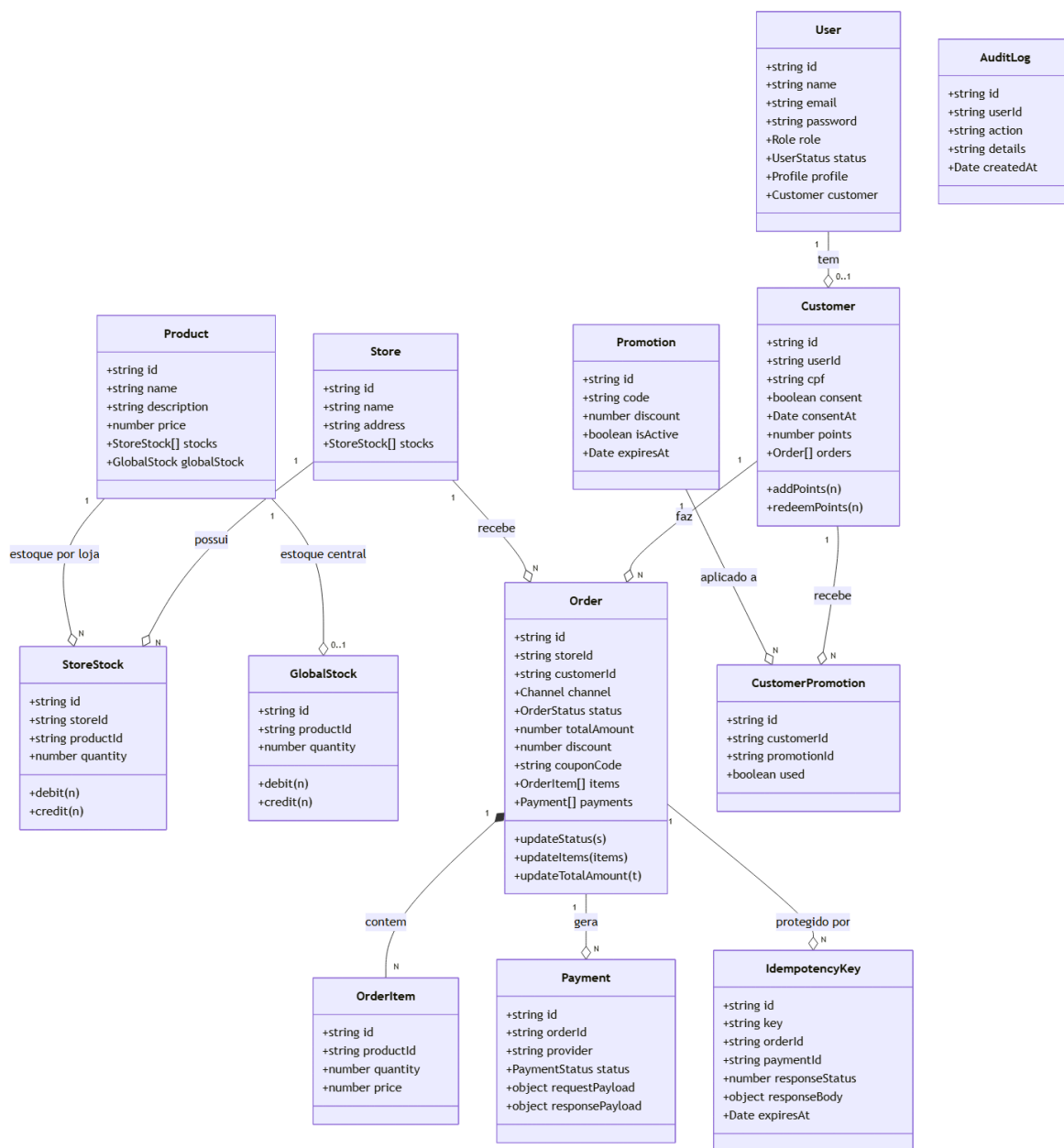
Cardinalidade	Entidades Envolvidas	Restricao Principal
1:1	User e Customer	Customer.userId UNIQUE.
1:N	User e RefreshToken	RefreshToken.hashToken UNIQUE; cascade on delete.
1:N	User e PasswordReset	PasswordReset.hashToken UNIQUE; cascade on delete.
1:N	Customer e Order	Order.customerId FK; cascade on delete definido.
1:N	Customer e CustomerPromotion	Par (customerId,promotionId) UNIQUE.
1:N	Store e Order	Order.storeId FK; toda Order pertence a uma Store.
1:N	Store e StoreStock	Par (storeId,productId) UNIQUE em StoreStock.
1:N	Product e StoreStock	Combinacao de estoque por unidade.
1:1	Product e GlobalStock	GlobalStock.productId UNIQUE.
1:N	Order e OrderItem	OrderItem.price congelado no momento da criacao.
1:N	Order e Payment	Payment desacoplado do Order; request/response JSON.
1:N	Order e IdempotencyKey	IdempotencyKey.key UNIQUE e TTL 24h.
1:N	Promotion e CustomerPromotion	Cada promocao pode ser vinculada a varios clientes.

3.3 Diagrama de Classes (Domínio)

O diagrama de classes da Figura 5 apresenta as principais entidades/objetos de domínio e seus relacionamentos. As classes aqui representadas são usadas diretamente nas camadas Application e Infrastructure, seguindo a organização de Clean Architecture.

Figura 5 - Diagrama de Classes do Domínio

[Clique para visualizar em alta resolução](#)

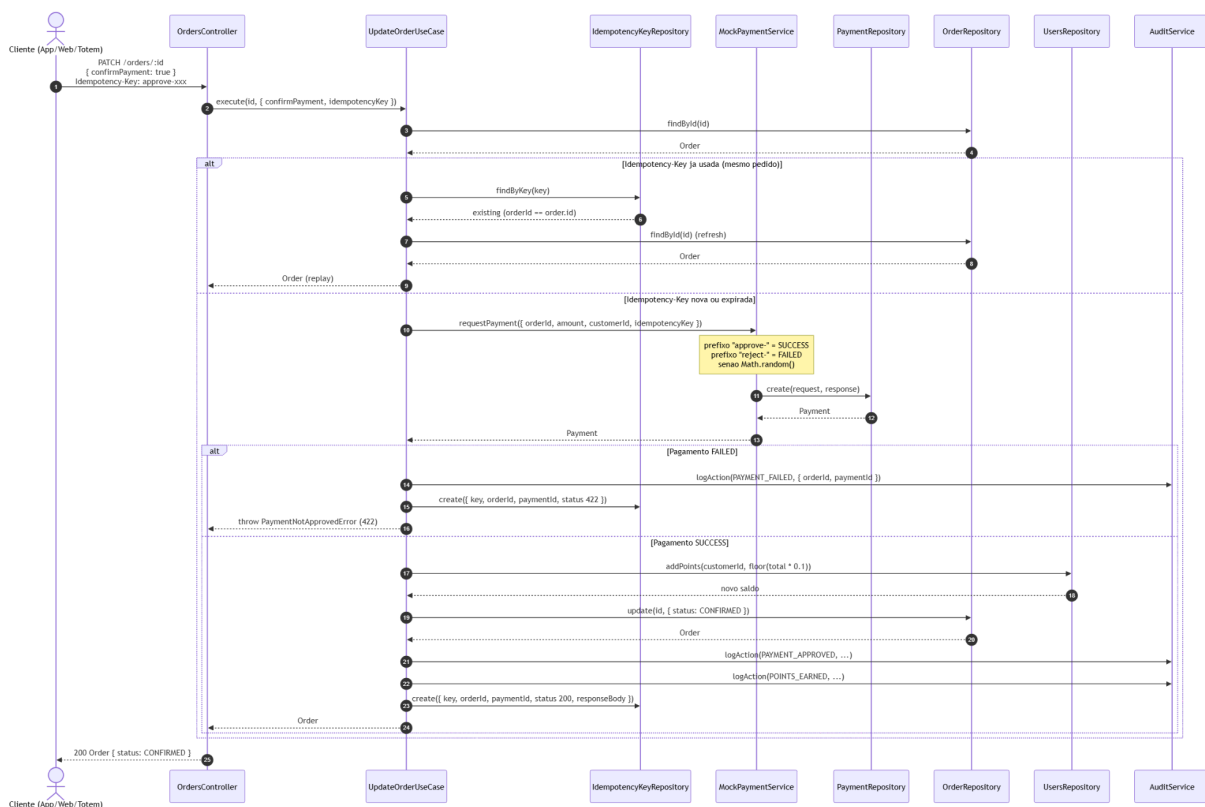


3.4 Diagrama de Sequência do Fluxo Crítico

A Figura 6 apresenta o diagrama de sequência do fluxo crítico Pedido -> Pagamento mock -> Atualização de status, com destaque para o tratamento da chave de idempotencia e dos cenários de pagamento aprovado/recusado.

Figura 6 - Diagrama de Sequência do Fluxo Crítico

[Clique para visualizar em alta resolução](#)

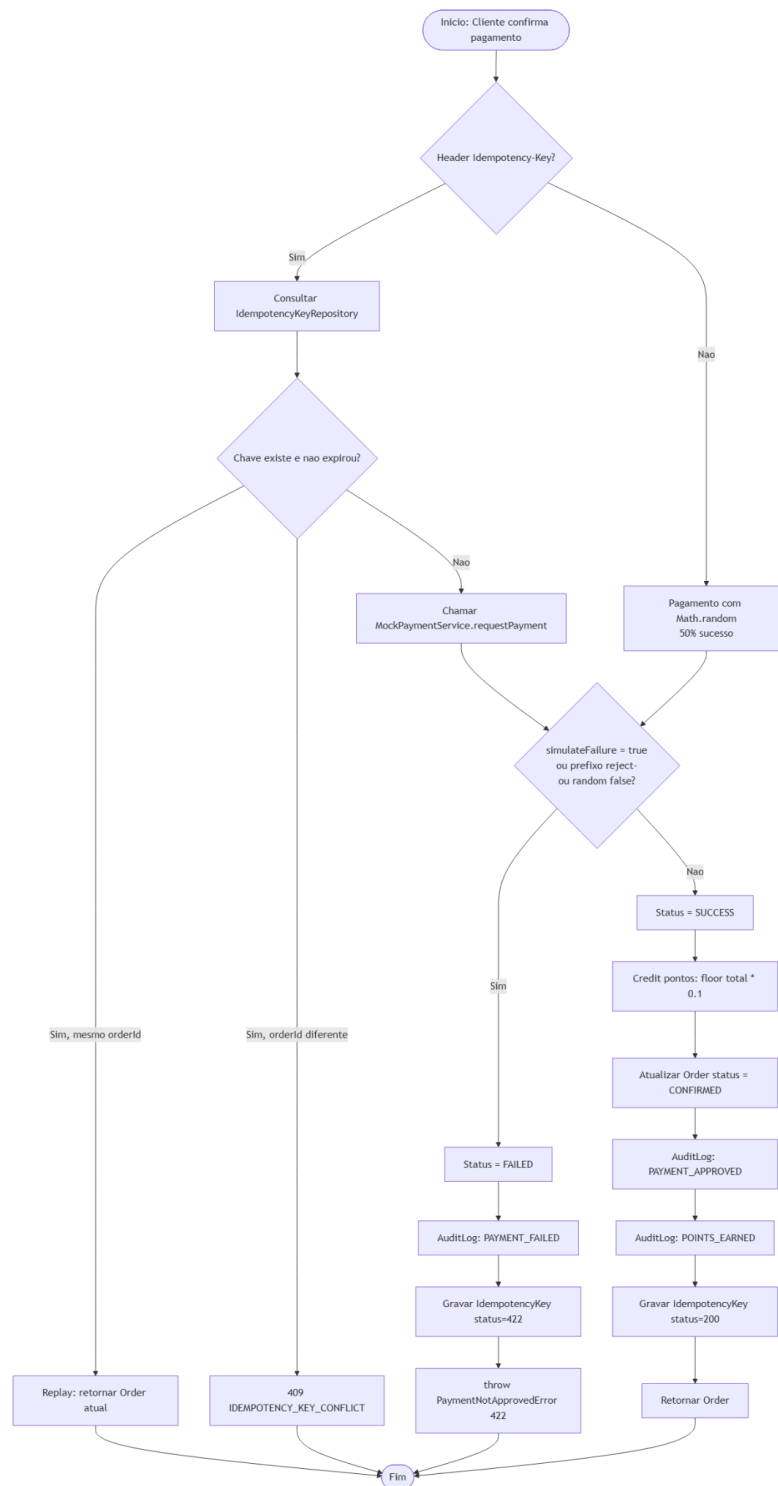


3.5 Diagrama de Atividade do Fluxo Crítico

A Figura 7 complementa o diagrama de sequência com um diagrama de atividade, evidenciando os nós de decisão relacionados a existência da chave de idempotencia e ao resultado do pagamento mock.

Figura 7 - Diagrama de Atividade do Fluxo Crítico

[Clique para visualizar em alta resolução](#)

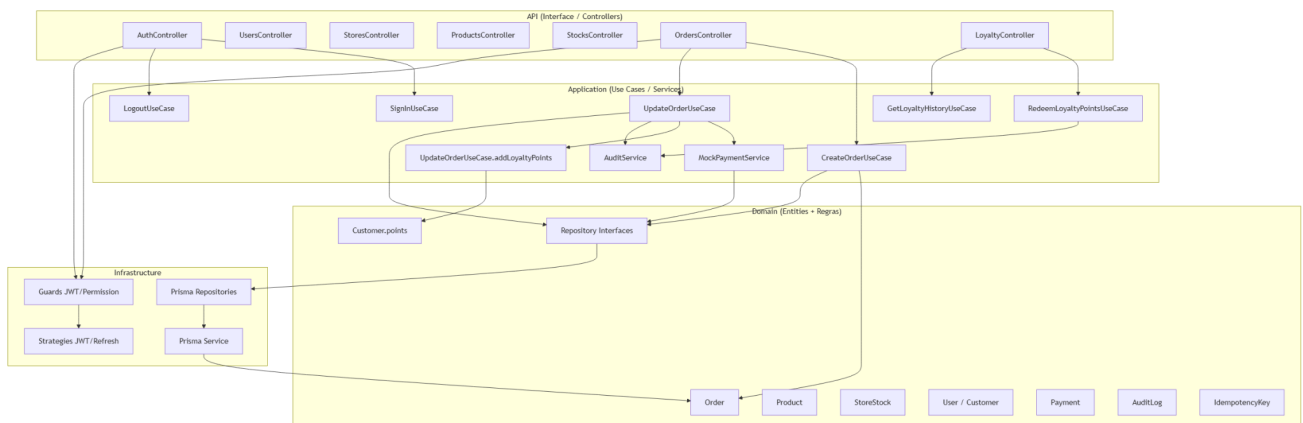


3.6 Arquitetura em Camadas

A organização do projeto segue a separação de responsabilidades recomendada pelo roteiro, em quatro camadas: Domain (entidades e regras), Application (casos de uso e serviços), Infrastructure (Prisma, Guards, Strategies) e API (Controllers). A Figura 8 representa essa organização.

Figura 8 - Arquitetura em Camadas

[Clique para visualizar em alta resolução](#)



Cada módulo de domínio da camada modules/ segue o mesmo padrão de organização interna:

- domain/ - entidades (caso aplicável), tipos e interfaces de repositório;
- application/ - casos de uso (use-cases) e serviços, com tratamento de erros específicos;
- infra/ - implementações HTTP (controllers, DTOs, view models) e persistência (repositórios Prisma).

4 API E ENDPOINTS

Esta seção descreve o contrato da API REST do projeto, agrupando os endpoints por recurso. Cada endpoint é apresentado com método HTTP, rota, autenticação/permissões, parâmetros, exemplo de request/response, códigos de status e participação no padrão de erro. A especificação completa, com tipos, decorators e exemplos, está disponível via Swagger em /docs (e também no arquivo docs/openapi.json no repositório).

A organização por recurso da API segue o seguinte padrão: /auth, /users, /stores, /products, /stocks, /orders, /loyalty, /promotions. Os endpoints seguem as regras mínimas exigidas pelo roteiro, quais sejam: URLs no plural, IDs no path, paginação em listagens (page, limit), códigos de status coerentes (200, 201, 204, 400/422, 401, 403, 404, 409) e padrão de erro consistente.

4.1 Módulo de Autenticação

Quadro 2 - Endpoints de Autenticação

Metodo	Rota	Autenticacao	Descricao
POST	/auth/sign-in	publico	Autêntica usuário (e-mail+senha) e retorna access token + refresh token.
POST	/auth/refresh-token	refresh token	Renova o access token a partir do refresh token.
POST	/auth/logout	refresh token	Revoga o refresh token atual.
POST	/auth/password-reset	publico	Solicita redefinição de senha.
POST	/auth/reset-password	publico	Confirma redefinição de senha com token.

Exemplo de login (POST /auth/sign-in):

```
Request: { "email": "cliente@exemplo.com", "senha": "Senha@123" }
Response 200: { "accessToken": "jwt...", "refreshToken": "jwt...",
"tokenType": "Bearer", "expiresIn": 3600, "user": { ... } }
```

Status codes esperados: 200, 400, 401, 422.

4.2 Módulo de Usuários

Quadro 3 - Endpoints de Usuários

Método	Rota	Autenticação	Descrição
POST	/users	publico	Cadastro público de cliente (com CPF e consentimento LGPD).
POST	/users/management	JWT + permissão	Criação de usuário administrativo/staff.
GET	/users	JWT + permissão	Lista usuários com paginação.
GET	/users/:id	JWT	Detalhe de usuário.
PATCH	/users/:id	JWT + permissão	Atualização parcial.
DELETE	/users/:id	JWT + permissão	Remoção.

O cadastro publico exige o objeto customerData com o consentimento explicito para o tratamento de dados pessoais. O CPF e validado quanto a formato e unicidade. A senha e armazenada como hash bcrypt.

4.3 Modulo de Unidades (Stores)

Quadro 4 - Endpoints de Unidades

Metodo	Rota	Autenticação	Descrição
POST	/stores	JWT + permissão	Cria uma unidade.
GET	/stores	publico	Lista unidades.
GET	/stores/:id	publico	Detalhe da unidade.
GET	/stores/:id/menu	publico	Cardápio da unidade, com disponibilidade (qtd) de cada item no estoque.
PUT	/stores/:id	JWT + permissão	Atualiza a unidade.
DELETE	/stores/:id	JWT + permissão	Remove a unidade.

O endpoint /stores/:id/menu retorna o cardápio com a quantidade disponível em estoque, permitindo que o cliente visualize somente itens passíveis de serem pedidos (requisito de disponibilidade do roteiro).

4.4 Modulo de Produtos, Estoques e Pedidos

Quadro 5 - Endpoints de Produtos e Estoques

Método	Rota	Autenticação	Descrição
POST	/products	JWT + permissão	Cria um produto.
GET	/products	JWT + permissão	Lista produtos.
GET	/products/:id	JWT + permissão	Detalhe do produto.
PATCH	/products/:id	JWT + permissão	Atualiza produto.
DELETE	/products/:id	JWT + permissão	Remove produto.
GET	/stocks/global	JWT + permissão	Lista estoque global.

POST	/stocks/global/:productId/transfer	JWT + permissão	Transfere N unidades do estoque global para uma loja.
GET	/stocks/store/:storeId	JWT + permissão	Lista estoque de uma loja.
PATCH	/stocks/store/:storeId/:productId	JWT + permissão	Ajusta quantidade na loja.

Quadro 6 - Endpoints de Pedidos (Fluxo Crítico)

Método	Rota	Autenticação	Descrição
POST	/orders	JWT	Cria pedido com channel obrigatório; valida estoque por unidade; debita StoreStock; congela preço nos itens.
GET	/orders	JWT	Lista pedidos com filtros (channel, status, storeId, customerId) e paginação.
GET	/orders/:id	JWT	Detalhe do pedido.
PATCH	/orders/:id	JWT	Atualiza status ou confirma pagamento (com header Idempotency-Key opcional).
DELETE	/orders/:id	JWT	Cancela pedido.

Exemplo de criação de pedido (POST /orders):

Request:

```
{
  "storeId": "bbbbbbbbb-bbbb-4bbb-8bbb-bbbbbbbbbbbb",
  "customerId": "uuid-do-cliente",
  "channel": "totem",
  "items": [
    { "productId": "id-produto", "quantity": 2 }
  ],
  "couponCode": "NORDESTE10" // opcional
}
```

Response 201:

```
{
  "id": "uuid-pedido",
  "status": "pending",
  "channel": "totem",
  "totalAmount": 79.80,
  "discount": 7.98,
```

```

    "items": [...],
    "createdAt": "2026-06-28T10:00:00Z"
  }

```

Status codes esperados: 201, 401, 403, 404, 409 (estoque insuficiente), 422 (channel ausente ou payload inválido).

Exemplo de confirmação de pagamento (PATCH /orders/:id com Idempotency-Key):

```

PATCH /orders/:id
Idempotency-Key: approve-pedido-9001
Content-Type: application/json

```

```

{ "confirmPayment": true }

```

```

Response 200: { id, status: "delivered", pointsCredited: 7, ... }

```

Status codes esperados: 200, 401, 403, 404, 409 (IDEMPOTENCY_KEY_CONFLICT ou PAYMENT_FAILED).

4.5 Modulo de Fidelidade (Loyalty)

Quadro 7 - Endpoints de Fidelidade

Método	Rota	Autenticação	Descrição
GET	/loyalty/balance	JWT	Consulta o saldo atual de pontos do cliente.
POST	/loyalty/redeem	JWT + consentimento	Resgate simples; exige consent=true; gera cupom.
GET	/loyalty/history	JWT	Extrato de movimentações (EARN, REDEEM, ORDER PAID).

Caso o cliente não possua consent=true, o endpoint POST /loyalty/redeem retorna 403 FORBIDDEN, sendo necessário atualizar o consentimento antes do resgate.

4.6 Modulo de Promoções

Quadro 8 - Endpoints de Promoções

Método	Rota	Autenticacao	Descricao
POST	/promotions	JWT + permissão	Cria uma promoção (código, desconto, validade).

GET	/promotions	JWT + permissão	Lista promoções.
GET	/promotions/:id	JWT + permissão	Detalhe.
PATCH	/promotions/:id	JWT + permissão	Atualização.
DELETE	/promotions/:id	JWT + permissão	Desativação.

A aplicação do cupom se dá no momento da criação do pedido (campo couponCode). Quando aplicado, o desconto é persistido no pedido e a promoção é marcada como usada para aquele cliente (CustomerPromotion.used = true), impedindo reuso.

4.7 padrões de Erro e Modulo de Auditoria

Todas as respostas de erro seguem o mesmo formato JSON, garantindo consistência no tratamento pelos clientes da API:

```
{
  "error": "NOME_DO_ERRO",          // código identificador
  "message": "Mensagem legível",    // mensagem para o usuário
  "details": [                      // detalhes de validação, opcional
    { "field": "campo", "issue": "problema" }
  ],
  "timestamp": "2026-06-28T12:00:00Z",
  "path": "/rota",                  // rota que retornou o erro
  "requestId": "uuid-opcional"      // correlação do request
}
```

Os principais códigos são: VALIDATION_ERROR (400/422), CREDENCIAIS_INVALIDAS (401), SEM_PERMISSAO (403), RECURSO_NAO_ENCONTRADO (404), REGRA_NEGOCIO (409), ESTOQUE_INSUFICIENTE (409), IDEMPOTENCY_KEY_CONFLICT (409), PAYMENT_FAILED (409), LGPD_CONSENT_REQUIRED (403).

A auditoria é exposta apenas internamente via tabela AuditLog; as ações sensíveis são registradas com userId, action e details (JSON). Exemplos de actions: ORDER_CREATED, ORDER_STATUS_UPDATED, ORDER_CANCELLED, PAYMENT_APPROVED, PAYMENT_FAILED, POINTS_EARNED, POINTS_REDEEMED.

5 LGPD, PRIVACIDADE E SEGURANCA NO BACK-END

A Lei Geral de Proteção de Dados (Lei n. 13.709/2018) é aplicada na prática no projeto, conforme demonstrado a seguir.

5.1 Dados pessoais coletados

Os dados pessoais coletados no cadastro de clientes são: nome, e-mail, CPF, senha (apenas hash) e o consentimento explícito. Também são armazenados, para fins de fidelidade, o saldo de pontos e o histórico de movimentações (earn, redeem, order_paid). Não são coletados dados sensíveis (saúde, origem racial, opinião política, etc.).

5.2 Finalidade e base legal

Os dados são tratados com a finalidade de execução do contrato de serviço (compra de produtos e gestão do programa de fidelidade), sendo a base legal a execução de contrato e, no caso de fidelidade, o consentimento explícito (LGPD, art. 7º, V e VII).

5.3 Como o consentimento é registrado

O modelo Customer possui os campos consent (boolean, obrigatório para resgate de pontos) e consentAt (timestamp). No cadastro público (/users), o payload deve incluir customerData.consent = true; sem isso, o registro não é concluído. O resgate (/loyalty/redeem) valida o consentimento e retorna 403 LGPD_CONSENT_REQUIRED caso o cliente não o tenha concedido.

5.4 Armazenamento seguro

Todas as senhas são armazenadas como hash bcrypt, com salt configurado por variável de ambiente (SALT_ROUNDS). Os tokens JWT são assinados com segredo forte (JWT_SECRET / JWT_ACCESS_SECRET) e têm tempos de expiração curtos (padrão 1h para access, 7d para refresh). Os refresh tokens são persistidos como hash, permitindo sua revogação por logout ou expiração.

5.5 Não exposição de dados sensíveis

As respostas da API são entregues por View Models, garantindo que campos como password, refreshToken em texto plano e dados pessoais não autorizados não sejam serializados. Os logs de auditoria não contêm senhas nem dados excessivos, apenas o userId do autor e detalhes da operação.

5.6 Logs/Auditoria

A tabela AuditLog registra todas as ações sensíveis: criação de pedido, mudança de status, cancelamento, pagamento aprovado/recusado, pontos ganhos/resgatados. Esses registros podem ser usados em análises posteriores pelo time de operações, sendo base para detecção de anomalias e para atendimento a solicitações de titulares (LGPD, art. 18).

5.7 Retenção e anonimização

A estratégia adotada no projeto consiste em: (i) manter o registro enquanto a relação contratual persistir; (ii) após o pedido de exclusão pelo titular, realizar a anonimização dos dados pessoais que não sejam necessários para cumprimento de obrigação legal; (iii) os registros de auditoria são retidos por período suficiente para fins de rastreabilidade, mas não contêm dados pessoais diretamente identificáveis além do userId.

6 ENTREGA TECNICA

Esta seção descreve os itens da entrega tecnica obrigatoria, conforme exigido pelo roteiro da Atividade Prática.

6.1 Repositório Git

O código-fonte do projeto está disponibilizado no repositório <https://github.com/LucasSousa15/raizes-do-nordeste-server>. O histórico de commits registra a evolução do trabalho em marcos como: feat: add env example and update readme; feat: add idempotency; feat: refine and structure audit and promotional modules; feat: adapt existent modules to attend specific reqs; feat: implement e2e tests. Cada commit tem mensagem no padrão Conventional Commits, facilitando o rastreamento histórico.

6.2 README de Execucao

O README.md do projeto orienta o usuário desde a instalação das dependências até a execução dos testes, com a sequência abaixo:

- Requisito prévio: Node.js 24+ e Docker (para PostgreSQL/Redis).
- Clonar o repositório e instalar dependências: `npm install`.
- Copiar o arquivo de variáveis: `cp .env.example .env` e editar credenciais locais.
- Subir o banco: `docker compose up -d`.
- Rodar migrations: `npx prisma migrate deploy` (ou `prisma migrate dev` em desenvolvimento).
- Popular o banco (opcional): `npm run db:seed` (cria 2 lojas, 10 produtos nordestinos e 3 promocoos).
- Iniciar a API: `npm run start:dev`. A aplicacao sobe em `http://localhost:3000`.
- Acessar a documentação Swagger: `http://localhost:3000/docs`.
- Rodar os testes: `npm run test` (unitários) e `npm run test:e2e` (end-to-end).

6.3 Documentação da API (Swagger/OpenAPI)

A documentação OpenAPI é gerada automaticamente pelo decorator `@nestjs/swagger`. Ela reflete fielmente os endpoints implementados, com seus DTOs tipados, exemplos de

request/response e os códigos de status. A especificação exportada está em docs/openapi.json, no repositório. O acesso via Swagger UI em /docs permite também testar as rotas (com autenticação Bearer).

6.4 Coleção de Testes Postman/Insomnia

A coleção exportada está em docs/postman/raizes-do-nordeste.postman_collection.json. Ela contém 25 cenários (T01 a T25), organizados em pastas (Auth, Users, Stores, Orders, Payments, Loyalty, Promotions, Erros), com exemplos de request/response e uso do header Idempotency-Key. As variáveis de ambiente (baseUrl, tokens) são gerenciadas automaticamente pela própria coleção.

6.5 Banco de Dados e Modelo (DER)

O DER do banco está em docs/DER.png (imagem) e em docs/diagrams.md (fonte Mermaid). A modelagem foi implementada no Prisma (prisma/schema.prisma) e migrada para PostgreSQL, de modo que o banco relacional e a API operam sobre o mesmo modelo.

6.6 Linha de comando e qualidade de código

Os utilitários npm principais são: build, start, start:dev, start:debug, start:prod, lint, test, test:cov, test:watch, test:e2e, db:seed, generate:module. O projeto adota ESLint + Prettier para manter a padronização, além de tipagem estrita TypeScript.

7 PLANO DE TESTES

O plano de testes foi estruturado em duas camadas: testes automatizados (unitários + e2e) e coleção executável (Postman/Insomnia). Abaixo estão consolidados os 25 cenários do roteiro, cada um com pré-condição, entrada, saída esperada e evidencia.

7.1 Cenários Mínimos (T01 a T25)

Quadro 9 - Plano de Testes - 25 cenários

ID	Cenário	Metodo/Rota	Saida Esperada
T01	Login valido	POST /auth/sign-in	200 + accessToken
T02	Login senha invalida	POST /auth/sign-in	401 CREDENCIAIS_INVALIDAS
T03	Cadastro cliente valido	POST /users	201 + User; consentimento registrado
T04	Acesso sem token	GET /orders	401 SEM_AUTENTICACAO
T05	Acesso sem permissão (cliente -> admin)	GET /products rota admin	403 SEM_PERMISSAO
T06	Cardapio por unidade	GET /stores/:id/menu	200 + items com availableQuantity
T07	Criar pedido valido sem cupom	POST /orders	201 + order pending; sem desconto
T08	Pedido sem canal	POST /orders	422 VALIDATION_ERROR (channel obrigatorio)
T09	Estoque insuficiente	POST /orders	409 ESTOQUE_INSUFICIENTE
T10	Pagamento mock aprovado	PATCH /orders/:id confirmPayment:true	200 + status delivered; 10% pontos creditados
T11	Pagamento mock recusado	PATCH /orders/:id simulatePaymentFailure:true	409 PAYMENT_FAILED; status anterior mantido
T12	Atualizar status manual	PATCH /orders/:id	200 + novo status
T13	Cancelar pedido	DELETE /orders/:id	204
T14	Consultar saldo fidelidade	GET /loyalty/balance	200 + points
T15	Resgatar pontos (gera cupom)	POST /loyalty/redeem	200 + couponCode + remainingPoints
T16	Resgate sem consentimento	POST /loyalty/redeem	403 LGPD_CONSENT_REQUIRED
T17	Criar pedido com cupom valido	POST /orders couponCode: ...	201 + discount > 0; totalAmount reduzido
T18	Criar pedido com cupom invalido/expirado	POST /orders couponCode: invalido	400 CUPOM_INVALIDO
T19	Cupom já utilizado	POST /orders couponCode: usado	400 CUPOM_JA_UTILIZADO

T20	Logs de auditoria criados	SELECT * FROM "AuditLog"	Registros ORDER_CREATED, PAYMENT_APPROVED , ORDER_STATUS_UPDATED
T21	Logout revoga refresh token	POST /auth/logout	204; refresh subsequente retorna 401
T22	Histórico de pontos	GET /loyalty/history	200 + lista EARN/REDEEM/ORDER PAID
T23	Mesma chave de idempotencia no mesmo pedido	PATCH /orders/:id Idempotency-Key 2x	200 + 1 unico Payment em DB
T24	Chave com prefixo reject-	PATCH /orders/:id reject-...	409 PAYMENT_FAILED; IdempotencyKey gravada
T25	Chave reutilizada em pedidos diferentes	PATCH orders distintos mesma chaves	409 IDEMPOTENCY_KEY_CONFLICT

7.2 Cobertura dos Requisitos Obrigatorios

Quadro 10 - Cobertura minima do roteiro

Requisito	Cenarios Cobertos
autenticaçãoe autorizacao	T01, T02, T04, T05
Validacao de dados	T08 (channel ausente), T18 (cupom invalido)
Regra de negocio (estoque/pedido)	T07, T09, T19
Pagamento mock aprovado/recusado	T10, T11, T24
Multicanalidade (canalPedido)	T07 + filtro GET /orders?channel=VALOR
Fidelidade + consentimento + resgate com cupom	T14, T15, T16
Cupons (aplicacao automatica)	T17, T18, T19
Logs/Auditoria	T20
Logout (revogacao)	T21
Historico de fidelidade	T22
Idempotencia no pagamento	T23, T24, T25

7.3 Testes automatizados

Alem da colecao Postman, o projeto conta com 125 testes unitarios e 22 testes e2e implementados em Vitest. Os testes unitarios usam repositorios in-memory por modulo, garantindo isolamento e velocidade. Os testes e2e sobem o contexto completo da aplicacao, conectam-se ao banco .env.test (isolado), executam migrations no globalSetup e truncam tabelas antes de cada suite. A execucao ocorre com:

```
npm run test          # 125 testes unitarios
npm run test:e2e      # 22 testes e2e
```

8 CONCLUSAO

O projeto Raizes do Nordeste atingiu os objetivos da Atividade Prática Multidisciplinar - Trilha Back-End 2026, entregando um backend completo para uma rede de lanchonetes nordestinas fictícia. A solução suporta os múltiplos canais exigidos (App, Web, Totem, Balcao e Pickup), com o canalPedido tratado como atributo obrigatório de domínio do pedido, garantindo rastreabilidade operacional.

O fluxo crítico escolhido foi o Pedido -> Pagamento mock -> Atualização de status, fechado ponta a ponta com persistência real em PostgreSQL e Prisma ORM. Acrescentou-se um mecanismo robusto de idempotencia (header Idempotency-Key com TTL de 24 horas e comportamento determinístico por prefixo da chave), essencial para evitar cobranças duplicadas em retentivas. As regras de validação seguem um padrão de erro consistente em todas as rotas, retornando códigos HTTP coerentes (200, 201, 204, 400/422, 401, 403, 404, 409) e o payload padrão definido no roteiro.

A modelagem de dados segue Clean Architecture em quatro camadas (API, Application, Domain, Infrastructure), com 15 entidades, cardinalidades explícitas e restrições de unicidade que refletem as regras de negócio (preço congelado em OrderItem, estoque próprio por unidade em StoreStock, pagamento desacoplado em Payment, idempotencia via IdempotencyKey, fidelidade e auditoria em módulos separados). O DER foi derivado diretamente do schema Prisma, garantindo coerência entre modelagem, código e banco.

Os requisitos de LGPD foram atendidos na prática: consentimento explícito no cadastro e no resgate de pontos, hash bcrypt de senhas, não exposição de dados sensíveis via View Models, e registros de auditoria para ações sensíveis. A separação de papéis (ADMIN, STAFF, CUSTOMER) e a autorização por Guards asseguram o controle de acesso.

A entrega técnica contempla o repositório Git, o README reproduzível, a documentação Swagger/OpenAPI em /docs, a coleção Postman executável com 25 cenários, a imagem do DER e o plano de testes completo. Além disso, o projeto é coberto por 125 testes unitários e 22 testes

e2e (com banco isolado via `.env.test`), demonstrando maturidade técnica e confiabilidade da implementação.

Como proposta de evolução, está documentada a aplicação automática de promoções no momento da criação do pedido (já parcialmente suportada), o refinamento do documento LGPD (finalidade, base legal, retenção) e a unificação final do padrão de erro com o template literal sugerido no roteiro. Em suma, o backend atende ao MVP da atividade, com vários módulos já amadurecidos além do escopo mínimo, mostrando capacidade de evolução para um cenário real de mercado.

REFERENCIAS

- BRASIL. Lei n. 13.709, de 14 de agosto de 2018. Lei Geral de Protecao de Dados Pessoais (LGPD). Brasilia, DF: Presidencia da Republica, 2018. Disponivel em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/113709.htm. Acesso em: 28 jun. 2026.
- ASSOCIACAO BRASILEIRA DE NORMAS TECNICAS. ABNT NBR 6023: informacao e documentacao - referencias - elaboracao. Rio de Janeiro: ABNT, 2018.
- ASSOCIACAO BRASILEIRA DE NORMAS TECNICAS. ABNT NBR 14724: informacao e documentacao - trabalhos academicos - apresentacao. Rio de Janeiro: ABNT, 2011.
- FOWLER, Martin. Patterns of enterprise application architecture. Boston: Addison-Wesley, 2002.
- GAMMA, Erich et al. Design patterns: elements of reusable object-oriented software. Reading, MA: Addison-Wesley, 1995.
- VERNON, Vaughn. Implementing domain-driven design. Boston: Addison-Wesley, 2013.
- MARTIN, Robert C. Clean architecture: a craftsman's guide to software structure and design. Boston: Prentice Hall, 2017.
- NESTJS. NestJS - A progressive Node.js framework. Disponivel em: <https://docs.nestjs.com/>. Acesso em: 28 jun. 2026.
- PRISMA. Prisma ORM documentation. Disponivel em: <https://www.prisma.io/docs>. Acesso em: 28 jun. 2026.
- NODE.JS FOUNDATION. Node.js Documentation. Disponivel em: <https://nodejs.org/en/docs>. Acesso em: 28 jun. 2026.
- JWT.IO. Introduction to JSON Web Tokens. Disponivel em: <https://jwt.io/introduction>. Acesso em: 28 jun. 2026.
- OPENAPI INITIATIVE. OpenAPI Specification. Disponivel em: <https://swagger.io/specification/>. Acesso em: 28 jun. 2026.
- UNINTER. Roteiro de Atividade Pratica de Projeto Multidisciplinar - Trilha Back-End 2026. Universidade Tuiuti do Parana, 2026.
- POSTMAN. Postman API Platform Documentation. Disponivel em: <https://learning.postman.com/>. Acesso em: 28 jun. 2026.
- VITEST. Vitest - A blazing fast unit test framework. Disponivel em: <https://vitest.dev/>. Acesso em: 28 jun. 2026.